



Sketch-Based Anomaly Detection in Streaming Graphs

Siddharth Bhatia
TurboML
siddharth@turboml.com

Mohit Wadhwa
mailmohitwadhwa@gmail.com

Kenji Kawaguchi
National University of Singapore
kenji@comp.nus.edu.sg

Neil Shah
Snap Inc.
nshah@snap.com

Philip S. Yu
University of Illinois at Chicago
psyu@uic.edu

Bryan Hooi
National University of Singapore
bhooi@comp.nus.edu.sg

ABSTRACT

Given a stream of graph edges from a dynamic graph, how can we assign anomaly scores to edges and subgraphs in an online manner, for the purpose of detecting unusual behavior, using constant time and memory? For example, in intrusion detection, existing work seeks to detect either anomalous edges or anomalous subgraphs, but not both. In this paper, we first extend the count-min sketch data structure to a higher-order sketch. This higher-order sketch has the useful property of preserving the dense subgraph structure (dense subgraphs in the input turn into dense submatrices in the data structure). We then propose 4 online algorithms that utilize this enhanced data structure, which (a) detect both edge and graph anomalies; (b) process each edge and graph in constant memory and constant update time per newly arriving edge, and; (c) outperform state-of-the-art baselines on 4 real-world datasets. Our method is the first streaming approach that incorporates dense subgraph search to detect graph anomalies in constant memory and time.

CCS CONCEPTS

• Computing methodologies → Anomaly detection.

KEYWORDS

Anomaly Detection, Dynamic Graphs, Stream, Sketch

ACM Reference Format:

Siddharth Bhatia, Mohit Wadhwa, Kenji Kawaguchi, Neil Shah, Philip S. Yu, and Bryan Hooi. 2023. Sketch-Based Anomaly Detection in Streaming Graphs. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '23)*, August 6–10, 2023, Long Beach, CA, USA. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3580305.3599504>

1 INTRODUCTION

Consider an intrusion detection system, in which anomalous behavior can be described as an individual or a group of attackers making a large number of connections to some set of targeted machines to restrict accessibility or look for potential vulnerabilities. We can model this as a dynamic graph, where nodes correspond to machines, and each edge represents a timestamped connection from one machine to another.

In this graph, edge anomalies include individual connections that are significantly more malicious than the rest of the connections in the network. In addition, anomalous behavior often also takes the form of a dense subgraph which could represent a group of malicious nodes that are communicating with each other in a way that is unusual compared to the rest of the graph, as shown in several real-world datasets in [1–3]. Detecting both these edge and subgraph anomalies together provides valuable insights into the structure and behavior of the network and can help identify trends or patterns that might not be evident by considering only one type of anomaly.

Similarly, in a financial system, edge anomalies might include transactions that are significantly larger or more frequent than the rest of the transactions in the system. Subgraph anomalies might include groups of individuals or businesses who are significantly more likely to be involved in fraudulent activity than the rest of the system. Identifying both types of anomalies simultaneously can help detect fraudulent activity and protect against financial loss.

Thus, we ask the question: Given a stream of graph edges from a dynamic graph, how can we assign anomaly scores to both edges and subgraphs in an online manner, for the purpose of detecting unusual behavior, using constant memory and constant update time per newly arriving edge?

Several approaches [4–10] aim to detect anomalies in graph settings. However, these approaches focus on static graphs, whereas many real-world graphs are time-evolving in nature. In streaming or online graph scenarios, some methods can detect the presence of anomalous edges, [3, 11–13], while others can detect anomalous subgraphs [1, 2, 14]. However, all existing methods are limited to either anomalous edge or graph detection but not able to detect both kinds of anomalies, as summarized in Table 1. As we discuss in Section 7, our approach outperforms existing methods in both accuracy and running time; and on both anomalous edge and subgraph detection scenarios. Moreover, our approach is the only streaming method that makes use of dense subgraph search to detect graph anomalies while only requiring constant memory and time.

We first extend the two-dimensional sketch to a *higher-order sketch* to enable it to embed the relation between the source and destination nodes in a graph. A higher-order sketch has the useful property of preserving the dense subgraph structure; dense subgraphs in the input turn into dense submatrices in this data structure. Thus, the problem of detecting a dense subgraph from a large graph reduces to finding a dense submatrix in a constant size matrix, which can be achieved in constant time. The higher-order sketch allows us to propose several algorithms to detect both



This work is licensed under a Creative Commons Attribution International 4.0 License.

KDD '23, August 6–10, 2023, Long Beach, CA, USA
© 2023 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0103-0/23/08.
<https://doi.org/10.1145/3580305.3599504>

anomalous edges and subgraphs in a streaming manner. We introduce two edge anomaly detection methods, ANOEDGE-G, and ANOEDGE-L, and two graph anomaly detection methods ANOGRAPH, and ANOGRAPH-K, that use the same data structure to detect the presence of a dense submatrix, and consequently anomalous edges, or subgraphs respectively. All our approaches process edges and graphs in constant time, and are independent of the graph size, i.e., they require constant memory. We also provide theoretical guarantees on the higher-order sketch estimate and the submatrix density measure. In summary, the main contributions of our paper are:

- (1) **Higher-Order Sketch (Section 4):** We transform the dense subgraph detection problem into finding a dense submatrix (which can be achieved in constant time) by extending the count-min sketch (CMS) data structure to a higher-order sketch.
- (2) **Streaming Anomaly Detection (Sections 5,6):** We propose four novel online approaches to detect anomalous edges and graphs in real-time, with constant memory and update time. Moreover, this is the first streaming work that incorporates dense subgraph search to detect graph anomalies in constant memory/time.
- (3) **Effectiveness (Section 7):** We outperform all state-of-the-art streaming edge and graph anomaly detection methods on four real-world datasets.

Reproducibility: Our code and datasets are available on <https://github.com/Stream-AD/AnoGraph>.

2 RELATED WORK

Our work is closely related to areas like anomaly detection on graphs [15–25] and streams [26–35], and streaming algorithms [36–40]. Higher-order sketches are discussed in [40], however, they are restricted to count-sketches and non-graph settings.

[41–43] discuss deep learning based anomaly detection and [44–48] discuss dense subgraph discovery, however, such approaches are unable to detect anomalies in a streaming manner. [4–10] are limited to anomaly detection in static graphs. In this section, however, we limit our review only to methods detecting edge and graph anomalies in dynamic graphs; see [49] for an extensive survey.

Edge Stream Methods: HOTSPOT [50] detects nodes whose egonets suddenly change. RHSS [51] focuses on sparsely-connected graph parts. CAD [52] localizes anomalous changes using commute time distance measurement. More recently, DENSESTREAM [1] maintains and updates a dense submatrix in a tensor stream. SEDANSPOT [11] identifies edge anomalies based on edge occurrence, preferential attachment, and mutual neighbors. PENminer [12] explores the persistence of activity snippets, i.e., the length and regularity of edge-update sequences' reoccurrences. F-FADE [13] aims to detect anomalous interaction patterns by factorizing their frequency. MIDAS [3, 53] identifies microcluster-based anomalies. However, all these methods are unable to detect graph anomalies.

Graph Stream Methods: DTA/STA [54] approximates the adjacency matrix of the current snapshot using matrix factorization. COPYCATCH [55] spots near-bipartite cores where each node is connected to others in the same core densely within a short time. SPOT/DSPOT [33] use extreme value theory to automatically set thresholds for anomalies. IncGM+ [56] utilizes incremental method

to process graph updates. More recently, DENSEALERT identifies subgraphs created within a short time and utilizes incremental method to process graph updates or subgraphs more efficiently. SPOTLIGHT [2] discovers anomalous graphs with dense bi-cliques, but uses a randomized approach without any search for dense subgraphs. ANOMRANK [14], inspired by PageRank [57], iteratively updates two score vectors and computes anomaly scores. However, these methods are slow and do not detect edge anomalies. Moreover, they do not search for dense subgraphs in constant memory/time.

3 PROBLEM

Let $\mathcal{E} = \{e_1, e_2, \dots\}$ be a stream of weighted edges from a time-evolving graph $\mathcal{G}$. Each arriving edge is a tuple $e_i = (u_i, v_i, w_i, t_i)$ consisting of a source node $u_i \in \mathcal{V}$, a destination node $v_i \in \mathcal{V}$, a weight w_i , and a time of occurrence t_i , the time at which the edge is added to the graph. For example, in a network traffic stream, an edge e_i could represent a connection made from a source IP address u_i to a destination IP address v_i at time t_i . We do not assume that the set of vertices $\mathcal{V}$ is known a priori: for example, new IP addresses or user IDs may be created over the course of the stream.

We model $\mathcal{G}$ as a directed graph. Undirected graphs can be handled by treating an incoming undirected edge as two simultaneous directed edges, one in each direction. We also allow $\mathcal{G}$ to be a multi-graph: edges can be created multiple times between the same pair of nodes. Edges are allowed to arrive simultaneously: i.e. $t_{i+1} \geq t_i$, since in many applications t_i is given as a discrete time tick.

The desired properties of our algorithm are as follows:

- **Detecting Anomalous Edges:** To detect whether the edge is part of an anomalous subgraph in an online manner. Being able to detect anomalies at the finer granularity of edges allows early detection so that recovery can be started as soon as possible and the effect of malicious activities is minimized.
- **Detecting Anomalous Graphs:** To detect the presence of an unusual subgraph (consisting of edges received over a period of time) in an online manner, since such subgraphs often correspond to unexpected behavior, such as coordinated attacks.
- **Constant Memory and Update Time:** To ensure scalability, memory usage and update time should not grow with the number of nodes or the length of the stream. Thus, for a newly arriving edge, our algorithm should run in constant memory and update time.

4 HIGHER-ORDER SKETCH & NOTATIONS

Count-min sketches (CMS) [58] are popular streaming data structures used by several online algorithms [59]. CMS uses multiple hash functions to map events to frequencies, but unlike a hash table uses only sub-linear space, at the expense of overcounting some events due to collisions. Frequency is approximated as the minimum over all hash functions. CMS, shown in Figure 1(a), is represented as a two-dimensional matrix where each row corresponds to a hash function and hashes to the same number of buckets (columns).

We introduce a Higher-order CMS (H-CMS) data structure where each hash function maps multi-dimensional input to a generic tensor instead of mapping it to a row vector. H-CMS enhances CMS by separately hashing the individual components of an entity thereby

Table 1: Comparison of relevant anomaly detection approaches.

Property	DenseStream (KDD'17)	SedanSpot (ICDM'20)	MIDAS-R (AAAI'20)	PENminer (KDD'20)	F-FADE (WSDM'21)	DenseAlert (KDD'17)	SpotLight (KDD'18)	AnomRank (KDD'19)	Our Method
Edge Anomaly	✓	✓	✓	✓	✓	–	–	–	✓
Graph Anomaly	–	–	–	–	–	✓	✓	✓	✓
Constant Memory	–	✓	✓	–	✓	–	✓	–	✓
Constant Update Time	–	✓	✓	✓	✓	–	✓	–	✓
Dense Subgraph Search	✓	–	–	–	–	✓	–	–	✓

maintaining more information. Figure 1(b) shows a 3-dimensional H-CMS that can be used to hash two-dimensional entities such as graph edges to a matrix. The source node is hashed to the first dimension and the destination node to the other dimension of the sketch matrix, as opposed to the original CMS that will hash the entire edge to a one-dimensional row vector (Figure 1(a)).

We use a 3-dimensional H-CMS (operations described in Algorithm 1) where the number of hash functions is denoted by n_r , and matrix M_j corresponding to j -th hash function h_j is of dimension $n_b \times n_b$, i.e., a square matrix. For each $j \in [n_r]$, the j -th hash function denoted by $h_j(u, v)$ maps an edge (u, v) to a matrix index $(h'_j(u), h''_j(v))$, i.e., the source node is mapped to a row index and the destination node is mapped to a column index. That is, $h_j(u, v) = (h'_j(u), h''_j(v))$. Therefore, each matrix in a 3-dimensional H-CMS captures the essence of a graph adjacency

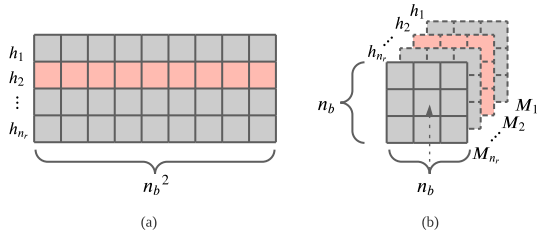


Figure 1: (a) Original CMS with n_b^2 buckets for each hash function (b) Higher-order CMS with $n_b \times n_b$ buckets for each hash function.

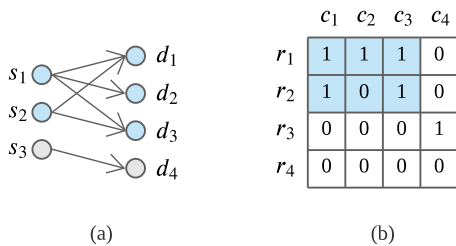


Figure 2: (a) Dense subgraph in the original graph between source nodes s_1, s_2, s_3 and destination nodes d_1, d_2, d_3 is transformed to a Dense submatrix between rows r_1, r_2 , and columns c_1, c_2, c_3 in the H-CMS (b).

matrix. Dense subgraph detection can thus be transformed into a dense submatrix detection problem (as shown in Figure 2) where the size of the matrix is a small constant, independent of the number of edges or the graph size.

For any (u, v) , let $y(u, v)$ be the true count of (u, v) observed thus far and $\hat{y}(u, v) = \min_{j \in [n_r]} M_j[h'_j(u)][h''_j(v)]$ be the estimate of the count via the 3-dimensional H-CMS. Since the H-CMS can overestimate the count by possible collisions (but not underestimate because we update and keep all the counts for every hash function), we have $y(u, v) \leq \hat{y}(u, v)$. We define M to be the number of all observations so far; i.e., $M = \sum_{u,v} y(u, v)$. The following theorem shows that the 3-dimensional H-CMS has the estimate guarantees similarly to the CMS:

THEOREM 1. (Proof in Appendix A) For all $k \in [n_r]$, let $h_k(u, v) = (h'_k(u), h''_k(v))$ where each of hash functions h'_k and h''_k is chosen uniformly at random from a pairwise-independent family. Here, we allow both cases of $h' = h''$ and $h' \neq h''$. Fix $\delta > 0$ and set $n_r = \lceil \ln \frac{1}{\delta} \rceil$ and $n_b = \lceil \frac{\epsilon}{\delta} \rceil$. Then, with probability at least $1 - \delta$, $\hat{y}(u, v) \leq y(u, v) + \epsilon M$.

Theorem 1 shows that we have the estimate guarantee even if we use the same hash function for both the source nodes and the destination node (i.e., $h' = h''$). Thus, with abuse of notation, we write $h(u, v) = (h(u), h(v))$ when $h' = h''$ by setting $h = h' = h''$ on the right-hand side. On the other hand, in the case of $h' \neq h''$,

Algorithm 1: H-CMS Operations

```

1 Procedure INITIALIZE H-CMS ( $n_r, n_b$ )
2   for  $r \leftarrow 1 \dots n_r$  do
3      $h_r : \mathcal{V} \rightarrow [0, n_b)$  // hash vertex
4      $M_r \leftarrow [0]_{n_b \times n_b}$ 

1 Procedure RESET H-CMS ( $n_r, n_b$ )
2   for  $r \leftarrow 1 \dots n_r$  do
3      $M_r \leftarrow [0]_{n_b \times n_b}$  // reset to zero matrix

1 Procedure UPDATE H-CMS ( $u, v, w$ )
2   for  $r \leftarrow 1 \dots n_r$  do
3      $M_r[h_r(u)][h_r(v)] \leftarrow M_r[h_r(u)][h_r(v)] + w$ 

1 Procedure DECAY H-CMS ( $\alpha$ )
2   for  $r \leftarrow 1 \dots n_r$  do
3      $M_r \leftarrow \alpha * M_r$  // decay factor:  $\alpha$ 

```

Table 2: Table of symbols.

Symbol	Definition
n_r	number of hash functions
n_b	number of buckets
$h(u)$	hash function $u \rightarrow [0, n_b)$
$\mathcal{M}$ $\mathcal{M}[i][j]$	a square matrix of dimensions $n_b \times n_b$ element at row index i and column index j
S	set of all row indices
S_{cur}	set of current submatrix row indices
S_{rem}	set of remaining row indices
T	set of all column indices
T_{cur}	set of current submatrix column indices
T_{rem}	set of remaining column indices
$[z]$	set of all integers in the range $[1, z]$
$\mathcal{D}(\mathcal{M}, S_x, T_x)$	density of submatrix (S_x, T_x)
$\mathcal{E}(\mathcal{M}, S_x, T_x)$	sum of elements of submatrix (S_x, T_x)
$\mathcal{R}(\mathcal{M}, u, T_x)$	submatrix row-sum i.e. sum of elements of submatrix $(\{u\}, T_x)$
$\mathcal{C}(\mathcal{M}, S_x, v)$	submatrix column-sum i.e. sum of elements of submatrix $(S_x, \{v\})$
$\mathcal{L}(\mathcal{M}, u, v, S_x, T_x)$	likelihood of index (u, v) w.r.t. submatrix (S_x, T_x)
d_{max}	maximum reported submatrix density

it would be possible to improve the estimate guarantee in Theorem 1. For example, if we can make h to be chosen uniformly at random from a weakly universal set of hash functions (by defining corresponding families of distributions for h' and h'' under some conditions), then we can set $n_b = \lceil \sqrt{\frac{\epsilon}{\epsilon}} \rceil$ to have the same estimate guarantee as that of Theorem 1 based on the proof of Theorem 1. The analysis for such a potential improvement is left for future work as an open problem.

Frequently used symbols are discussed in Table 2, and we leverage the subgraph density measure discussed in [60] to define the submatrix (S_x, T_x) density.

DEFINITION 1. Given matrix $\mathcal{M}$, density of a submatrix of $\mathcal{M}$ represented by $S_x \subseteq S$ and $T_x \subseteq T$, is:

$$\mathcal{D}(\mathcal{M}, S_x, T_x) = \frac{\sum_{s \in S_x} \sum_{t \in T_x} \mathcal{M}[s][t]}{\sqrt{|S_x| |T_x|}} \quad (1)$$

5 EDGE ANOMALIES

In this section, using the H-CMS data structure, we propose ANOEDGE-G and ANOEDGE-L to detect edge anomalies by checking whether the received edge when mapped to a sketch matrix element is part of a dense submatrix. ANOEDGE-G finds a Global dense submatrix and ANOEDGE-L maintains and updates a Local dense submatrix around the matrix element.

5.1 ANOEDGE-G

ANOEDGE-G, as described in Algorithm 2, maintains a *temporally decaying* H-CMS, i.e. whenever 1 unit of time passes, we multiply all the H-CMS counts by a fixed factor α (lines 2,4). This decay simulates the gradual ‘forgetting’ of older, and hence, more out-dated information. When an edge (u, v) arrives, u, v are mapped to matrix indices $h(u), h(v)$ respectively for each hash function h , and the corresponding H-CMS counts are updated (line 5). EDGE-SUBMATRIX-DENSITY procedure (described below) is then called to compute the density of a dense submatrix around $(h(u), h(v))$. Density is reported as the anomaly score for the edge; a larger density implies that the edge is more likely to be anomalous.

EDGE-SUBMATRIX-DENSITY procedure calculates the density of a dense submatrix around a given index $(h(u), h(v))$. A 1×1 submatrix represented by S_{cur} and T_{cur} , is initialized with row-index $h(u)$ and column index $h(v)$ (line 9). The submatrix is iteratively expanded by greedily selecting a row u_p from S_{rem} (or a column v_p from T_{rem}) that obtains the maximum row (or column) sum with the current submatrix (lines 11,12). This selected row u_p (or column v_p) is removed from S_{rem} (or T_{rem}), and added to S_{cur} (or T_{cur}) (lines 14,16). The process is repeated until both S_{rem} and T_{rem} are empty (line 10). Density of the current submatrix is computed at each iteration of the submatrix expansion process and the maximum over all greedily formed submatrix densities is returned (lines 17,18).

Algorithm 2: ANOEDGE-G Scoring

Input: Stream $\mathcal{E}$ of edges over time

Output: Anomaly score per edge

```

1 Procedure ANOEDGE-G( $\mathcal{E}$ )
2   Initialize H-CMS matrix  $\mathcal{M}$  for edge count
3   while new edge  $e = (u, v, w, t) \in \mathcal{E}$  is received do
4     /* decay count */
5     Temporal decay H-CMS with timestamp change
6     Update H-CMS matrix  $\mathcal{M}$  for new edge  $(u, v)$  with
       value  $w$  // update count
7     output  $\text{score}(e) \leftarrow$ 
8       EDGE-SUBMATRIX-DENSITY( $\mathcal{M}, h(u), h(v)$ )
9
10 Procedure EDGE-SUBMATRIX-DENSITY( $\mathcal{M}, u, v$ )
11    $S \leftarrow [n_b]; T \leftarrow [n_b]; S_{cur} \leftarrow \{u\}; T_{cur} \leftarrow \{v\};$ 
12    $S_{rem} \leftarrow S/\{u\}; T_{rem} \leftarrow T/\{v\}$ 
13    $d_{max} \leftarrow \mathcal{D}(\mathcal{M}, S_{cur}, T_{cur})$ 
14   while  $S_{rem} \neq \emptyset \vee T_{rem} \neq \emptyset$  do
15     /* submatrix max row-sum index */
16      $u_p \leftarrow \arg\max_{s_p \in S_{rem}} \mathcal{R}(\mathcal{M}, s_p, T_{cur})$ 
17     /* submatrix max column-sum index */
18      $v_p \leftarrow \arg\max_{t_p \in T_{rem}} \mathcal{C}(\mathcal{M}, S_{cur}, t_p)$ 
19     if  $\mathcal{R}(\mathcal{M}, u_p, T_{cur}) > \mathcal{C}(\mathcal{M}, S_{cur}, v_p)$  then
20        $S_{cur} \leftarrow S_{cur} \cup \{u_p\}; S_{rem} \leftarrow S_{rem}/\{u_p\}$ 
21     else
22        $T_{cur} \leftarrow T_{cur} \cup \{v_p\}; T_{rem} \leftarrow T_{rem}/\{v_p\}$ 
23      $d_{max} \leftarrow \max(d_{max}, \mathcal{D}(\mathcal{M}, S_{cur}, T_{cur}))$ 
24   return  $d_{max}$  // dense submatrix density
```

PROPOSITION 1. (Proof in Appendix B.1) Time complexity of Algorithm 2 is $O(|\mathcal{E}| * n_r * n_b^2)^1$. Memory complexity of Algorithm 2 is $O(n_r * n_b^2)$.

5.2 ANOEDGE-L

Inspired by Definition 1, we define the likelihood measure of a matrix index $(h(u), h(v))$ with respect to a submatrix (S_x, T_x) , as the sum of the elements of submatrix (S_x, T_x) that either share row with index $h''(v)$ or column with index $h'(u)$ divided by the total number of such elements.

DEFINITION 2. Given matrix $\mathcal{M}$, likelihood of an index $h(u, v)$ with respect to a submatrix represented by $S_x \subseteq S$ and $T_x \subseteq T$, is:

$$\mathcal{L}(\mathcal{M}, u, v, S_x, T_x) = \frac{\sum_{(s,t) \in S_x \times \{h(v)\} \cup \{h(u)\} \times T_x} \mathcal{M}[s][t]}{|S_x \times \{h(v)\} \cup \{h(u)\} \times T_x|} \quad (2)$$

ANOEDGE-L, as described in Algorithm 3, maintains a temporally decaying H-CMS to store the edge counts. We also initialize a mutable submatrix of size 1×1 with a random element, and represent it as (S_{cur}, T_{cur}) . As we process edges, we greedily update (S_{cur}, T_{cur}) to maintain it as a dense submatrix. When an edge arrives, H-CMS counts are first updated, and the received edge is then used to check whether to *expand* the current submatrix (line 7). If the submatrix density increases upon the addition of the row (or column), then the row-index $h(u)$ (or column-index $h(v)$) is added to the current submatrix, (S_{cur}, T_{cur}) . To remove the row(s) and column(s) decayed over time, the process iteratively selects the row (or column) with the minimum row-sum (or column-sum) until removing it increases the current submatrix density. This ensures that the current submatrix is as *condensed* as possible (line 9). As defined in Definition 2, ANOEDGE-L computes the likelihood score of the edge with respect to (S_{cur}, T_{cur}) (line 10). A higher likelihood measure implies that the edge is more likely to be anomalous.

Algorithm 3: ANOEDGE-L Scoring

Input: Stream $\mathcal{E}$ of edges over time

Output: Anomaly score per edge

```

1 Procedure ANOEDGE-L( $\mathcal{E}$ )
2   Initialize H-CMS matrix  $\mathcal{M}$  for edges count
   /* mutable submatrix */
3   Initialize a randomly picked  $1 \times 1$  submatrix  $(S_{cur}, T_{cur})$ 
4   while new edge  $e = (u, v, w, t) \in \mathcal{E}$  is received do
   /* decay count */
5     Temporal decay H-CMS with timestamp change
6     Update H-CMS matrix  $\mathcal{M}$  for new edge  $(u, v)$  with
       value  $w$  // update count
7     ▷ Check and Update Submatrix:
8     Expand  $(S_{cur}, T_{cur})$  // expand submatrix
9     Condense  $(S_{cur}, T_{cur})$  // condense submatrix
   /* likelihood score from Definition 2 */
10    output score( $e$ )  $\leftarrow \mathcal{L}(\mathcal{M}, h(u), h(v), S_{cur}, T_{cur})$ 

```

¹This is for processing all edges; the time per edge is constant.

PROPOSITION 2. (Proof in Appendix B.2) Time complexity of Algorithm 3 is $O(n_r * n_b^2 + |\mathcal{E}| * n_r * n_b)$. Memory complexity of Algorithm 3 is $O(n_r * n_b^2)$.

6 GRAPH ANOMALIES

We now propose ANOGRAPH and ANOGRAPH-K to detect graph anomalies by first mapping the graph to a higher-order sketch, and then checking for a dense submatrix. These are the first streaming algorithms that make use of dense subgraph search to detect graph anomalies in constant memory and time. ANOGRAPH greedily finds a dense submatrix with a 2-approximation guarantee on the density measure. ANOGRAPH-K leverages EDGE-SUBMATRIX-DENSITY from Algorithm 2 to greedily find a dense submatrix around K strategically picked matrix elements performing equally well in practice.

6.1 ANOGRAPH

ANOGRAPH, as described in Algorithm 4, maintains an H-CMS to store the edge counts that are reset whenever a new graph arrives. The edges are first processed to update the H-CMS counts. ANOGRAPH-DENSITY procedure (described below) is then called to find the dense submatrix. ANOGRAPH reports anomaly score as the density of the detected (dense) submatrix; a larger density implies that the graph is more likely to be anomalous.

ANOGRAPH-DENSITY procedure computes the density of a dense submatrix of matrix $\mathcal{M}$. The current dense submatrix is initialised as matrix $\mathcal{M}$ and then the row (or column) from the current submatrix with minimum row (or column) sum is greedily removed. This process is repeated until S_{cur} and T_{cur} are empty (line 11). The density of the current submatrix is computed at each iteration of the submatrix expansion process and the maximum over all densities is returned (lines 18, 19).

Algorithm 4 is a special case of finding the densest subgraph in a directed graph problem [60] where the directed graph is represented as an adjacency matrix and detecting the densest subgraph essentially means detecting dense submatrix. We now provide a guarantee on the density measure.

LEMMA 1. Let S^* and T^* be the optimum densest sub-matrix solution of $\mathcal{M}$ with density $\mathcal{D}(\mathcal{M}, S^*, T^*) = d_{opt}$. Then $\forall u \in S^*$ and $\forall v \in T^*$,

$$\mathcal{R}(\mathcal{M}, u, T^*) \geq \tau_{S^*}; \quad \mathcal{C}(\mathcal{M}, S^*, v) \geq \tau_{T^*} \quad (3)$$

$$\text{where: } \tau_{S^*} = \mathcal{E}(\mathcal{M}, S^*, T^*) \left(1 - \sqrt{1 - \frac{1}{|S^*|}}\right),$$

$$\tau_{T^*} = \mathcal{E}(\mathcal{M}, S^*, T^*) \left(1 - \sqrt{1 - \frac{1}{|T^*|}}\right)$$

PROOF. Leveraging the proof from [60], let's assume that $\exists u \in S^*$ with $\mathcal{R}(\mathcal{M}, u, T^*) < \tau_{S^*}$. Density of submatrix after removing $u = \frac{\mathcal{E}(\mathcal{M}, S^*, T^*) - \mathcal{R}(\mathcal{M}, u, T^*)}{\sqrt{(|S^*|-1)|T^*|}}$ which is greater than $\frac{\mathcal{E}(\mathcal{M}, S^*, T^*) - \tau_{S^*}}{\sqrt{(|S^*|-1)|T^*|}} = d_{opt}$, and that is not possible. Hence, $\mathcal{R}(\mathcal{M}, u, T^*) \geq \tau_{S^*}$. $\mathcal{C}(\mathcal{M}, S^*, v) \geq \tau_{T^*}$ can be proved in a similar manner. $\square$

THEOREM 2. ANOGRAPH-DENSITY procedure in Algorithm 4 achieves a 2-approximation guarantee for the densest submatrix problem.

PROOF. Leveraging the proof from [60], we greedily remove the row (or column) with minimum row-sum (or column-sum). At some iteration of the greedy process, $\forall u \in S_{cur}; \forall v \in T_{cur}$, $\mathcal{R}(\mathcal{M}, u, T_{cur}) \geq \tau_{S^*}$ and $\mathcal{C}(\mathcal{M}, S_{cur}, v) \geq \tau_{T^*}$. Therefore, $\mathcal{E}(\mathcal{M}, S_{cur}, T_{cur}) \geq |S_{cur}| \tau_{S^*}$ and $\mathcal{E}(\mathcal{M}, S_{cur}, T_{cur}) \geq |T_{cur}| \tau_{T^*}$. This implies that the density $\mathcal{D}(\mathcal{M}, S_{cur}, T_{cur}) \geq \sqrt{\frac{|S_{cur}| \tau_{S^*} |T_{cur}| \tau_{T^*}}{|S_{cur}| |T_{cur}|}} = \sqrt{\tau_{S^*} \tau_{T^*}}$. Putting values of τ_{S^*} and τ_{T^*} from Lemma 1, and setting $|S^*| = \frac{1}{\sin^2 \alpha}$, $|T^*| = \frac{1}{\sin^2 \beta}$, we get $\mathcal{D}(\mathcal{M}, S_{cur}, T_{cur}) \geq \frac{\mathcal{E}(\mathcal{M}, S^*, T^*)}{\sqrt{|S^*| |T^*|}} \frac{\sqrt{(1-\cos \alpha)(1-\cos \beta)}}{\sin \alpha \sin \beta} \geq \frac{d_{opt}}{2 \cos \frac{\alpha}{2} \cos \frac{\beta}{2}} \geq \frac{d_{opt}}{2}$. $\square$

Algorithm 4: ANOGRAPHScoring

Input: Stream $\mathcal{G}$ of edges over time
Output: Anomaly score per graph

```

1 Procedure ANOGRAPH( $\mathcal{G}$ )
2   Initialize H-CMS matrix  $\mathcal{M}$  for graph edges count
3   while new graph  $G \in \mathcal{G}$  is received do
4     /* reset count */
5     Reset H-CMS matrix  $\mathcal{M}$  for graph  $G$ 
6     for edge  $e = (u, v, w, t) \in G$  do
7       Update H-CMS matrix  $\mathcal{M}$  for edge  $(u, v)$  with
         value  $w$  // update count
8     /* anomaly score */
9     output  $score(G) \leftarrow \text{ANOGRAPH-DENSITY}(\mathcal{M})$ 

10 Procedure ANOGRAPH-DENSITY( $\mathcal{M}$ )
11    $S_{cur} \leftarrow [n_b]$ ;  $T_{cur} \leftarrow [n_b]$  // initialize to size
    of  $\mathcal{M}$ 
12    $d_{max} \leftarrow \mathcal{D}(\mathcal{M}, S_{cur}, T_{cur})$ 
13   while  $S_{cur} \neq \emptyset \vee T_{cur} \neq \emptyset$  do
14     /* submatrix min row-sum index */
15      $u_p \leftarrow \text{argmin}_{s_p \in S_{cur}} \mathcal{R}(\mathcal{M}, s_p, T_{cur})$ 
16     /* submatrix min column-sum index */
17      $v_p \leftarrow \text{argmin}_{t_p \in T_{cur}} \mathcal{C}(\mathcal{M}, S_{cur}, t_p)$ 
18     if  $\mathcal{R}(\mathcal{M}, u_p, T_{cur}) < \mathcal{C}(\mathcal{M}, S_{cur}, v_p)$  then
19        $S_{cur} \leftarrow S_{cur} / \{u_p\}$  // remove row
20     else
21        $T_{cur} \leftarrow T_{cur} / \{v_p\}$  // remove column
22      $d_{max} \leftarrow \max(d_{max}, \mathcal{D}(\mathcal{M}, S_{cur}, T_{cur}))$ 
23   return  $d_{max}$  // dense submatrix density

```

PROPOSITION 3. (Proof in Appendix C.1) Time complexity of Algorithm 4 is $O(|\mathcal{G}| * n_r * n_b^2 + |\mathcal{G}| * n_r)$. Memory complexity of Algorithm 4 is $O(n_r * n_b^2)$.

6.2 ANOGRAPH-K

Similar to ANOGRAPH, ANOGRAPH-K maintains an H-CMS which is reset whenever a new graph arrives. It uses the ANOGRAPH-K-DENSITY procedure (described below) to find the dense submatrix. ANOGRAPH-K is summarised in Algorithm 5.

ANOGRAPH-K-DENSITY computes the density of a dense submatrix of matrix $\mathcal{M}$. The intuition comes from the heuristic that the matrix elements with a higher value are more likely to be part of a dense submatrix. Hence, the approach considers K largest elements of the matrix $\mathcal{M}$ and calls EDGE-SUBMATRIX-DENSITY from Algorithm 2 to get the dense submatrix around each of those elements (line 13). The maximum density over the considered K dense submatrices is returned.

Algorithm 5: ANOGRAPH-K Scoring

Input: Stream $\mathcal{G}$ of edges over time
Output: Anomaly score per graph

```

1 Procedure ANOGRAPH-K( $\mathcal{G}, K$ )
2   Initialize H-CMS matrix  $\mathcal{M}$  for graph edges count
3   while new graph  $G \in \mathcal{G}$  is received do
4     /* reset count */
5     Reset H-CMS matrix  $\mathcal{M}$  for graph  $G$ 
6     for edge  $e = (u, v, w, t) \in G$  do
7       Update H-CMS matrix  $\mathcal{M}$  for edge  $(u, v)$  with
         value  $w$  // update count
8     /* anomaly score */
9     output  $score(G) \leftarrow \text{ANOGRAPH-K-DENSITY}(\mathcal{M}, K)$ 

10 Procedure ANOGRAPH-K-DENSITY( $\mathcal{M}, K$ )
11    $B \leftarrow [n_b] \times [n_b]$  // set of all indices
12    $d_{max} \leftarrow 0$ 
13   for  $j \leftarrow 1 \dots K$  do
14     /* pick the max element */
15      $u_p, v_p \leftarrow \text{argmax}_{(s_p, t_p) \in B} \mathcal{M}[s_p][t_p]$ 
16      $d_{max} \leftarrow \max(d_{max}, \text{EDGE-SUBMATRIX-DENSITY}(\mathcal{M}, u_p, v_p))$ 
17    $B \leftarrow B / \{(u_p, v_p)\}$  // remove max element index
18   return  $d_{max}$  // dense submatrix density

```

PROPOSITION 4. (Proof in Appendix C.2) Time complexity of Algorithm 5 is $O(|\mathcal{G}| * K * n_r * n_b^2 + |\mathcal{G}| * n_r)$. Memory complexity of Algorithm 5 is $O(n_r * n_b^2)$.

7 EXPERIMENTS

Table 3: Statistics of the datasets.

Dataset	V	E	T
DARPA	25,525	4,554,344	46,567
ISCX-IDS2012	30,917	1,097,070	165,043
CIC-IDS2018	33,176	7,948,748	38,478
CIC-DDoS2019	1,290	20,364,525	12,224

In this section, we evaluate the performance of our approaches as compared to all baselines discussed in Table 1 and aim to answer the following questions:

Table 4: AUC and Running Time when detecting edge anomalies. Averaged over 5 runs.

Dataset	DENSESTREAM	SEDANSPOT	MIDAS-R	PENminer	F-FADE	ANOEDGE-G	ANOEDGE-L
DARPA	0.532 57.7s	0.647 $\pm$ 0.006 129.1s	0.953 $\pm$ 0.002 1.4s	0.872 5.21 hrs	0.919 $\pm$ 0.005 317.8s	0.970 $\pm$ 0.001 28.7s	0.964 $\pm$ 0.001 6.1s
ISCX-IDS2012	0.551 138.6s	0.581 $\pm$ 0.001 19.5s	0.820 $\pm$ 0.050 5.3s	0.530 1.3 hrs	0.533 $\pm$ 0.020 137.4s	0.954 $\pm$ 0.000 7.8s	0.957 $\pm$ 0.003 0.7s
CIC-IDS2018	0.756 3.3 hours	0.325 $\pm$ 0.037 209.6s	0.919 $\pm$ 0.019 1.1s	0.821 10 hrs	0.607 $\pm$ 0.001 279.7s	0.963 $\pm$ 0.014 58.4s	0.927 $\pm$ 0.035 10.2s
CIC-DDoS2019	0.263 265.6s	0.567 $\pm$ 0.004 697.6s	0.983 $\pm$ 0.003 2.2s	— > 24 hrs	0.717 $\pm$ 0.041 18.7s	0.997 $\pm$ 0.001 123.3s	0.998 $\pm$ 0.001 17.8s

Q1. Edge Anomalies: How accurately do ANOEDGE-G and ANOEDGE-L detect edge anomalies compared to baselines? Are they fast and scalable?

Q2. Graph Anomalies: How accurately do ANOGRAPH and ANOGRAPH-K detect graph anomalies i.e. anomalous graph snapshots? Are they fast and scalable?

Table 3 shows the statistical summary of the four real-world datasets that we use: *DARPA* [61] and *ISCX-IDS2012* [62] are popular datasets for graph anomaly detection used by baselines to evaluate their algorithms; [63] surveys more than 30 datasets and recommends to use the newer *CIC-IDS2018* and *CIC-DDoS2019* datasets [64, 65] containing modern attack scenarios. $|E|$ corresponds to the total number of edge records, $|V|$ and $|T|$ are the number of unique nodes and unique timestamps, respectively.

Similar to baseline papers, we report the Area under the ROC curve (AUC) and the running time. AUC is calculated by plotting the true positive rate (TPR) against the false positive rate (FPR) at various classification thresholds and then calculating the area under the resulting receiver operating characteristic (ROC) curve. The appropriate classification threshold for an anomaly detection system will depend on the specific application and the cost of false positives and false negatives, however, since AUC is independent of the classification threshold, one can evaluate the overall performance of the system without having to choose a specific threshold. Unless explicitly specified, all experiments including those on the baselines are repeated 5 times and the mean is reported.

Appendix D describes the experimental setup. Hyperparameters for the baselines are provided in Appendix E. All edge (or graph)-based methods output an anomaly score per edge (or graph), a higher score implying more anomalousness.

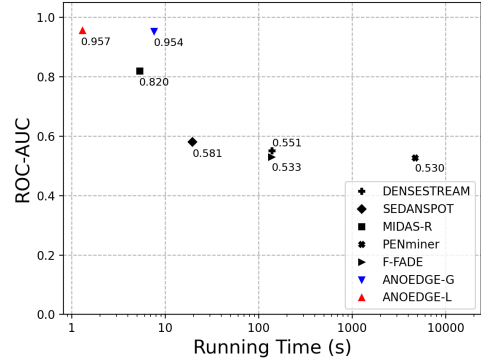
7.1 Edge Anomalies

Accuracy: Table 4 shows the AUC of edge anomaly detection baselines, ANOEDGE-G, and ANOEDGE-L. We report a single value for DENSESTREAM and PENminer because these are non-randomized methods. PENminer is unable to finish on the large *CIC-DDoS2019* within 24 hours. SEDANSPOT uses personalised PageRank to detect anomalies and is not always able to detect anomalous edges occurring in dense block patterns while PENminer is unable to detect structural anomalies. Among the baselines, MIDAS-R is most

accurate, however, it performs worse when there is a large number of timestamps as in *ISCX-IDS2012*. Note that ANOEDGE-G and ANOEDGE-L outperform all baselines on all datasets.

Running Time: Table 4 shows the running time (excluding I/O) and real-time performance of ANOEDGE-G and ANOEDGE-L. Since ANOEDGE-L maintains a local dense submatrix, it is faster than ANOEDGE-G. DENSESTREAM maintains dense blocks incrementally for every coming tuple and updates dense subtensors when it meets an updating condition, limiting the detection speed. SEDANSPOT requires several subprocesses (hashing, random-walking, reordering, sampling, etc), PENminer and F-FADE need to actively extract patterns for every graph update, resulting in a large computation time. When there is a large number of timestamps like in *ISCX-IDS2012*, MIDAS-R performs slower than ANOEDGE-L which is fastest.

AUC vs Running Time: Figure 3 plots accuracy (AUC) vs. running time (log scale, in seconds, excluding I/O) on *ISCX-IDS2012* dataset. ANOEDGE-G and ANOEDGE-L achieve much higher accuracy compared to all baselines, while also running significantly faster.

**Figure 3: AUC vs running time when detecting edge anomalies on *ISCX-IDS2012***

Scalability: Figures 4(a) and 4(b) plot the running time with increasing number of hash functions and edges respectively, on the *ISCX-IDS2012* dataset. This demonstrates the scalability of ANOEDGE-G and ANOEDGE-L.

ANOEDGE-G vs ANOEDGE-L: ANOEDGE-G finds a Global dense submatrix and therefore is more accurate than ANOEDGE-L as

Table 5: AUC and Running Time when detecting graph anomalies. Averaged over 5 runs.

Dataset	DENSEALERT	SPOTLIGHT	ANOMRANK	ANOGGRAPH	ANOGGRAPH-K
DARPA	0.833 49.3s	0.728 ± 0.016 88.5s	0.754 3.7s	0.835 ± 0.002 0.3s	0.839 ± 0.002 0.3s
ISCX-IDS2012	0.906 6.4s	0.872 ± 0.019 21.1s	0.194 5.2s	0.950 ± 0.001 0.5s	0.950 ± 0.001 0.5s
CIC-IDS2018	0.950 67.9s	0.835 ± 0.022 149.0s	0.783 7.0s	0.957 ± 0.000 0.2s	0.957 ± 0.000 0.3s
CIC-DDoS2019	0.764 1065.0s	0.468 ± 0.048 289.7s	0.241 0.2s	0.946 ± 0.002 0.4s	0.948 ± 0.002 0.4s

shown in the performance on *CIC-IDS2018*. ANOEDGE-L on the other hand maintains and updates a Local dense submatrix around the matrix element and therefore has better time complexity and scalability to larger datasets.

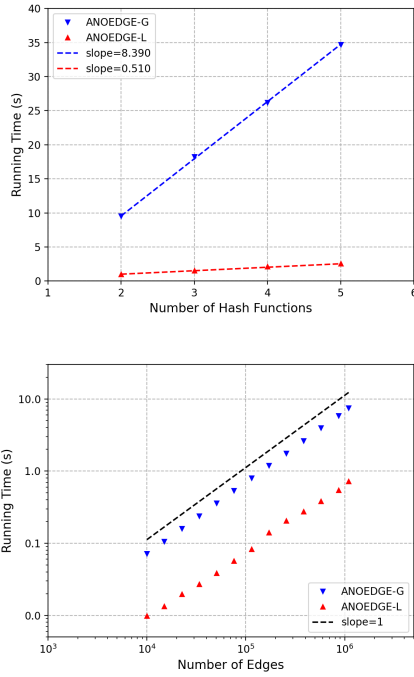


Figure 4: (a) Linear scalability with number of hash functions. (b) Linear scalability with number of edges.

7.2 Graph Anomalies

Accuracy: Table 5 shows the AUC of graph anomaly detection baselines, ANOGRAPH, and ANOGRAPH-K. We report a single value for DENSEALERT and ANOMRANK because these are non-randomized methods. ANOMRANK is not meant for a streaming scenario, therefore the low AUC. DENSEALERT can estimate only one subtensor at a time and SPOTLIGHT uses a randomized approach without any

actual search for dense subgraphs. Note that ANOGRAPH and ANOGRAPH-K outperform all baselines on all datasets while using a simple sketch data structure to incorporate dense subgraph search as opposed to the baselines.

Running Time: Table 5 shows the running time (excluding I/O). DENSEALERT has $O(|\mathcal{E}|)$ worse case time complexity (per incoming edge). ANOMRANK needs to compute a global PageRank, which does not scale for stream processing. Note that ANOGRAPH and ANOGRAPH-K run much faster than all baselines.

AUC vs Running Time: Figure 5 plots accuracy (AUC) vs. running time (log scale, in seconds, excluding I/O) on the *CIC-DDoS2019* dataset. ANOGRAPH and ANOGRAPH-K achieve much higher accuracy compared to the baselines, while also running significantly faster.

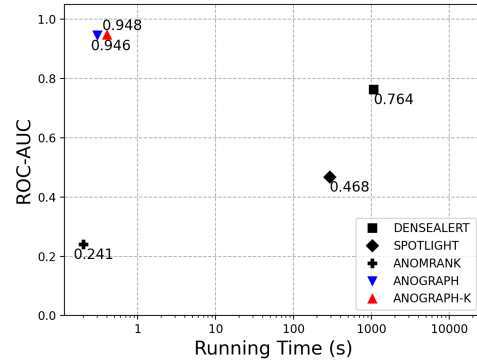


Figure 5: AUC vs running time when detecting graph anomalies on *CIC-DDoS2019*

Scalability: Figures 6(a), 6(b), and 6(c) plot the running time with increasing factor K (used for top- K in Algorithm 5), number of hash functions and number of edges respectively, on the *CIC-DDoS2019* dataset. This demonstrates the scalability of ANOGRAPH and ANOGRAPH-K.

ANOGRAPH vs ANOGRAPH-K: ANOGRAPH greedily finds a dense submatrix and has a time complexity of $O(|\mathcal{E}| * n_r * n_b^2 + |\mathcal{E}| * n_r)$, while ANOGRAPH-K greedily finds a dense submatrix around K strategically picked matrix elements and has a time complexity

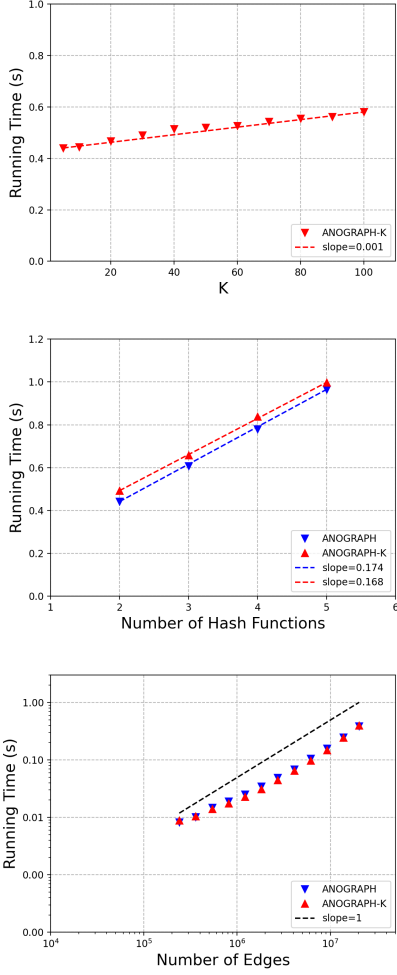


Figure 6: (a) ANOGRAPH-K scales linearly with factor K . (b) Linear scalability with number of hash functions. (c) Linear scalability with number of edges.

of $O(|\mathcal{G}| * K * n_r * n_b^2 + |\mathcal{G}| * n_r)$. Therefore, ANOGRAPH-K is faster when K is significantly smaller than n . ANOGRAPH-K is also more robust because it only considers a small number of matrix elements.

7.3 Hyperparameter Study

Table 6 shows the performance of ANOGRAPH and ANOGRAPH-K for multiple time windows and edge thresholds. The edge threshold is varied in such a way that a sufficient number of anomalies are present within the time window. ANOGRAPH and ANOGRAPH-K achieve comparable results to those in Table 5. Table 7 shows the robustness of ANOEDGE-G and ANOEDGE-L as we vary the temporal decay factor α .

8 CONCLUSION

In this paper, we extend the CMS data structure to a higher-order sketch to capture complex relations in graph data and to reduce the

Table 6: Influence of time window and edge threshold on the ROC-AUC when detecting graph anomalies.

Dataset	Time Window	Edge Threshold	ANOGRAPH	ANOGRAPH-K
DARPA	15	25	0.835 ± 0.001	0.838 ± 0.001
	30	50	0.835 ± 0.002	0.839 ± 0.002
	60	50	0.747 ± 0.002	0.748 ± 0.001
	60	100	0.823 ± 0.000	0.825 ± 0.001
ISCX-IDS2012	15	25	0.945 ± 0.001	0.945 ± 0.000
	30	50	0.949 ± 0.001	0.948 ± 0.000
	60	50	0.935 ± 0.002	0.933 ± 0.002
	60	100	0.950 ± 0.001	0.950 ± 0.001
CIC-IDS2018	15	25	0.945 ± 0.004	0.947 ± 0.006
	30	50	0.959 ± 0.000	0.959 ± 0.001
	60	50	0.920 ± 0.001	0.920 ± 0.001
	60	100	0.957 ± 0.000	0.957 ± 0.000
CIC-DDoS2019	15	25	0.864 ± 0.002	0.863 ± 0.003
	30	50	0.861 ± 0.003	0.861 ± 0.003
	60	50	0.824 ± 0.004	0.825 ± 0.005
	60	100	0.946 ± 0.002	0.948 ± 0.002

Table 7: Influence of temporal decay factor α on the ROC-AUC in ANOEDGE-G and ANOEDGE-L on DARPA.

α	ANOEDGE-G	ANOEDGE-L
0.2	0.964	0.957
0.4	0.966	0.959
0.6	0.968	0.961
0.8	0.969	0.964
0.9	0.969	0.966
0.95	0.966	0.966

problem of detecting suspicious dense subgraphs to finding a dense submatrix in constant time. We then propose four sketch-based streaming methods to detect edge and subgraph anomalies in constant update time and memory. Furthermore, our approach is the first streaming work that incorporates dense subgraph search to detect graph anomalies in constant memory and time. We also provide a theoretical guarantee on the submatrix density measure and prove the time and space complexities of all methods. Experimental results on four real-world datasets demonstrate our effectiveness as opposed to popular state-of-the-art streaming edge and graph baselines. Future work could consider incorporating rectangular H-CMS matrices, node and edge representations, more general types of data, including tensors, and parallel computing to process large dynamic graphs with a high volume of incoming edges.

ACKNOWLEDGMENTS

This work was supported by the National Research Foundation Singapore, NCS Pte. Ltd., National University of Singapore under the NUS-NCS Joint Laboratory (Grant A-0008542-00-00), and by NSF under Grant SaTC-1930941.

REFERENCES

- [1] K. Shin, B. Hooi, J. Kim, and C. Faloutsos, "Densealert: Incremental dense-subtensor detection in tensor streams," *KDD*, 2017.
- [2] D. Eswaran, C. Faloutsos, S. Guha, and N. Mishra, "Spotlight: Detecting anomalies in streaming graphs," in *KDD*, 2018.
- [3] S. Bhatia, B. Hooi, M. Yoon, K. Shin, and C. Faloutsos, "Midas: Microcluster-based detector of anomalies in edge streams," in *AAAI*, 2020.
- [4] L. Akoglu, M. McGlohon, and C. Faloutsos, "Oddball: Spotting anomalies in weighted graphs," in *PAKDD*, 2010.
- [5] D. Chakrabarti, "Autopart: Parameter-free graph partitioning and outlier detection," in *PKDD*, 2004.
- [6] B. Hooi, K. Shin, H. A. Song, A. Beutel, N. Shah, and C. Faloutsos, "Graph-based fraud detection in the face of camouflage," *TKDD*, 2017.
- [7] M. Jiang, P. Cui, A. Beutel, C. Faloutsos, and S. Yang, "Catching synchronized behaviors in large networks: A graph mining approach," *TKDD*, 2016.
- [8] J. M. Kleinberg, "Authoritative sources in a hyperlinked environment," *JACM*, 1999.
- [9] K. Shin, T. Eliassi-Rad, and C. Faloutsos, "Patterns and anomalies in k-cores of real-world graphs with applications," *KAIS*, 2018.
- [10] H. Tong and C.-Y. Lin, "Non-negative residual matrix factorization with application to graph anomaly detection," in *SDM*, 2011.
- [11] D. Eswaran and C. Faloutsos, "Sedanspot: Detecting anomalies in edge streams," in *ICDM*, 2018.
- [12] C. Belth, X. Zheng, and D. Koutra, "Mining persistent activity in continually evolving networks," in *KDD*, 2020.
- [13] Y.-Y. Chang, P. Li, R. Sosc, M. Affi, M. Schweighauser, and J. Leskovec, "F-fade: Frequency factorization for anomaly detection in edge streams," in *WSDM*, 2021.
- [14] M. Yoon, B. Hooi, K. Shin, and C. Faloutsos, "Fast and accurate anomaly detection in dynamic graphs with a two-pronged approach," in *KDD*, 2019.
- [15] J. Zhang, S. Liu, W. Yu, W. Feng, and X. Cheng, "Eigenpulse: Detecting surges in large streaming graphs with row augmentation," in *PAKDD*, 2019.
- [16] P. Bogdanov, C. Faloutsos, M. Mongiovi, E. E. Papalexakis, R. Ranca, and A. K. Singh, "Netspot: Spotting significant anomalous regions on dynamic networks," in *SDM*, 2013.
- [17] N. Shah, A. Beutel, B. Hooi, L. Akoglu, S. Gunnemann, D. Makhija, M. Kumar, and C. Faloutsos, "Edgecentric: Anomaly detection in edge-attributed networks," in *ICDMW*, 2016.
- [18] B. Perozzi and L. Akoglu, "Discovering communities and anomalies in attributed graphs: Interactive visual exploration and summarization," *TKDD*, 2018.
- [19] F. Bonchi, I. Bordino, F. Gullo, and G. Stilo, "The importance of unexpectedness: Discovering buzzing stories in anomalous temporal graphs," *Web Intelligence*, 2019.
- [20] F. Bonchi, I. Bordino, F. Gullo, and G. Stilo, "Identifying buzzing stories via anomalous temporal subgraph discovery," in *WI*, 2016.
- [21] A. Bojchevski and S. Gunnemann, "Bayesian robust attributed graph clustering: Joint learning of partial anomalies and group structure," in *AAAI*, 2018.
- [22] W. Yu, W. Cheng, C. Aggarwal, K. Zhang, H. Chen, and W. Wang, "Netwalk: A flexible deep embedding approach for anomaly detection in dynamic networks," *KDD*, 2018.
- [23] A. Kumagai, T. Iwata, and Y. Fujiwara, "Semi-supervised anomaly detection on attributed graphs," *IJCNN*, 2021.
- [24] Y. Liu, S. Pan, Y. G. Wang, F. Xiong, L. Wang, and V. C. Lee, "Anomaly detection in dynamic graphs via transformer," *ArXiv*, vol. abs/2106.09876, 2021.
- [25] M. Shao, J. Li, F. Chen, and X. Chen, "An efficient framework for detecting evolving anomalous subgraphs in dynamic networks," *INFOCOM*, 2018.
- [26] S. Bhatia, A. Jain, P. Li, R. Kumar, and B. Hooi, "Mstream: Fast anomaly detection in multi-aspect streams," *The Web Conference (WWW)*, 2021.
- [27] S. Bhatia, A. Jain, S. Srivastava, K. Kawaguchi, and B. Hooi, "Memstream: Memory-based anomaly detection in multi-aspect streams with concept drift," *The Web Conference (WWW)*, 2022.
- [28] E. A. Manzoor, H. Lamba, and L. Akoglu, "xstream: Outlier detection in feature-evolving data streams," in *KDD*, 2018.
- [29] S. C. Tan, K. M. Ting, and T. F. Liu, "Fast anomaly detection for streaming data," in *IJCAI*, 2011.
- [30] D. Jankov, S. Sikdar, R. Mukherjee, K. Teymourian, and C. Jermaine, "Real-time high performance anomaly detection over data streams: Grand challenge," *DEBS*, 2017.
- [31] S. Zou, Y. Liang, H. V. Poor, and X. Shi, "Nonparametric detection of anomalous data streams," *IEEE Transactions on Signal Processing*, 2017.
- [32] M. Moshtaghi, J. C. Bezdek, C. Leckie, S. Karunasekera, and M. Palaniswami, "Evolving fuzzy rules for anomaly detection in data streams," *IEEE Transactions on Fuzzy Systems*, 2015.
- [33] A. Siffer, P.-A. Fouque, A. Termier, C. Largouët, and C. Largouët, "Anomaly detection in streams with extreme value theory," *KDD*, 2017.
- [34] M. U. Togbe, M. Barry, A. Boly, Y. Chabchoub, R. Chiky, J. Montiel, and V.-T. Tran, "Anomaly detection for data streams based on isolation forest using scikit-multiflow," in *ICCSA*, 2020.
- [35] J. Zhang, S. Liu, W. Hou, S. Bhatia, H.-W. Shen, W. Yu, and X. Cheng, "Augsplicing: Synchronized behavior detection in streaming tensors," *AAAI*, 2021.
- [36] S. Pan, X. Zhu, C. Zhang, and S. Y. Philip, "Graph stream classification using labeled and unlabeled graphs," in *ICDE*, 2013.
- [37] W. Wang, X. Guan, and X. Zhang, "Processing of massive audit data streams for real-time anomaly intrusion detection," *Computer communications*, 2008.
- [38] A. K. Menon, G. V. A. Pham, S. Chawla, and A. Viglas, "An incremental data-stream sketch using sparse random projections," in *SDM*, 2007.
- [39] P. Zhao, C. C. Aggarwal, and M. Wang, "gsketch: On query estimation in graph streams," *Vldb*, 2011.
- [40] Y. Shi and A. Anandkumar, "Higher-order count sketch: Dimensionality reduction that retains efficient tensor operations," *DCC*, 2020.
- [41] R. Chalapathy and S. Chawla, "Deep learning for anomaly detection: A survey," *ArXiv*, vol. abs/1901.03407, 2019.
- [42] G. Pang, C. Shen, L. Cao, and A. van den Hengel, "Deep learning for anomaly detection: A review," *arXiv preprint arXiv:2007.02500*, 2020.
- [43] S. Bhatia, A. Jain, and B. Hooi, "Exgan: Adversarial generation of extreme samples," *AAAI Conference on Artificial Intelligence (AAAI)*, 2021.
- [44] C. Ma, Y. Fang, R. Cheng, L. Lakshmanan, W. Zhang, and X. Lin, "Efficient algorithms for densest subgraph discovery on large directed graphs," *SIGMOD*, 2020.
- [45] A. Epasto, S. Lattanzi, and M. Sozio, "Efficient densest subgraph computation in evolving graphs," *WWW*, 2015.
- [46] S. Sawlani and J. Wang, "Near-optimal fully dynamic densest subgraph," *STOC*, 2020.
- [47] A. McGregor, D. Tench, S. Vorotnikova, and H. Vu, "Densest subgraph in dynamic graph streams," *MFCS*, 2015.
- [48] H. Esfandiari and M. Mitzenmacher, "Metric sublinear algorithms via linear sampling," *FOCS*, 2018.
- [49] L. Akoglu, H. Tong, and D. Koutra, "Graph based anomaly detection and description: A survey," *Data mining and knowledge discovery*, 2015.
- [50] W. Yu, C. C. Aggarwal, S. Ma, and H. Wang, "On anomalous hotspot discovery in graph streams," in *ICDM*, 2013.
- [51] S. Ranshous, S. Harenberg, K. Sharma, and N. F. Samatova, "A scalable approach for outlier detection in edge streams using sketch-based approximations," in *SDM*, 2016.
- [52] K. Sricharan and K. Das, "Localizing anomalous changes in time-evolving graphs," in *SIGMOD*, 2014.
- [53] S. Bhatia, R. Liu, B. Hooi, M. Yoon, K. Shin, and C. Faloutsos, "Real-time anomaly detection in edge streams," *Transactions on Knowledge Discovery from Data (TKDD)*, 2022.
- [54] J. Sun, D. Tao, and C. Faloutsos, "Beyond streams and graphs: dynamic tensor analysis," in *KDD*, 2006.
- [55] A. Beutel, W. Xu, V. Guruswami, C. Palow, and C. Faloutsos, "Copycatch: stopping group attacks by spotting lockstep behavior in social networks," in *WWW*, 2013.
- [56] E. Abdelhamid, M. Canim, M. Sadoghi, B. Bhattacharjee, Y.-C. Chang, and P. Kalnis, "Incremental frequent subgraph mining on large evolving graphs," *TKDE*, 2017.
- [57] L. Page, S. Brin, R. Motwani, and T. Winograd, "The pagerank citation ranking : Bringing order to the web," in *WWW*, 1999.
- [58] G. Cormode and S. Muthukrishnan, "An improved data stream summary: the count-min sketch and its applications," *Journal of Algorithms*, 2005.
- [59] A. McGregor, "Graph stream algorithms: a survey," *SIGMOD Record*, 2014.
- [60] S. Khuller and B. Saha, "On finding dense subgraphs," in *ICALP*, 2009.
- [61] R. Lippmann, R. K. Cunningham, D. J. Fried, I. Graf, K. R. Kendall, S. E. Webster, and M. A. Zissman, "Results of the darpa 1998 offline intrusion detection evaluation," in *Recent advances in intrusion detection*, 1999.
- [62] A. Shiravi, H. Shiravi, M. Tavallaee, and A. A. Ghorbani, "Toward developing a systematic approach to generate benchmark datasets for intrusion detection," *computers & security*, 2012.
- [63] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," *Computers & Security*, 2019.
- [64] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *ICISSP*, 2018.
- [65] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (ddos) attack dataset and taxonomy," in *ICCS*, 2019.
- [66] R. C. Forest, <https://github.com/aws/random-cut-forest-by-aws>, 2021.
- [67] J. L. Carter and M. N. Wegman, "Universal classes of hash functions," *Journal of computer and system sciences*, 1979.

APPENDIX

A H-CMS ESTIMATE GUARANTEE PROOF

THEOREM 1. For all $k \in [n_r]$, let $h_k(u, v) = (h'_k(u), h''_k(v))$ where each of hash functions h'_k and h''_k is chosen uniformly at random from a pairwise-independent family. Here, we allow both cases of $h' = h''$ and $h' \neq h''$. Fix $\delta > 0$ and set $n_r = \lceil \ln \frac{1}{\delta} \rceil$ and $n_b = \lceil \frac{\epsilon}{\delta} \rceil$. Then, with probability at least $1 - \delta$, $\hat{y}(u, v) \leq y(u, v) + \epsilon M$.

PROOF. Fix $j \in [n_r]$. Let $a = (u_a, v_a)$ and $b = (u_b, v_b)$ such that $a \neq b$. This implies that at least one of the following holds: $u_a \neq u_b$ or $v_a \neq v_b$. Since h'_j (and h''_j) is chosen uniformly at random from a pairwise-independent family, $P(h'_j(u_a) = h'_j(u_b)) = \frac{1}{n_b}$ or $P(h''_j(v_a) = h''_j(v_b)) = \frac{1}{n_b}$. If $P(h'_j(u_a) = h'_j(u_b)) = \frac{1}{n_b}$, we have that $P(h_j(a) = h_j(b)) = P(h'_j(u_a) = h'_j(u_b) \wedge h''_j(v_a) = h''_j(v_b)) = P(h'_j(u_a) = h'_j(u_b))P(h''_j(v_a) = h''_j(v_b)) \leq \frac{1}{n_b} = \frac{\epsilon}{\delta}$. Similarly, if $P(h''_j(v_a) = h''_j(v_b)) = \frac{1}{n_b}$, $P(h_j(a) = h_j(b)) = P(h'_j(u_a) = h'_j(u_b) \wedge h''_j(v_a) = h''_j(v_b)) \leq \frac{1}{n_b} = \frac{\epsilon}{\delta}$. Thus, in the both cases, the probability of the collision is $P(h_j(a) = h_j(b)) = \frac{\epsilon}{\delta}$. Thus, by defining $X_{a,j} = \sum_b \mathbb{1}\{a \neq b \wedge h_j(a) = h_j(b)\} y(b)$, $\mathbb{E}[X_{a,j}] \leq \sum_b y(b) \mathbb{E}[\mathbb{1}\{a \neq b \wedge h_j(a) = h_j(b)\}] \leq \frac{\epsilon}{\delta} M$. Since $\hat{y}(a) = \min_j y(a) + X_{a,j}$, this implies that $P(\hat{y}(a) > y(a) + \epsilon M) = P(\min_j y(a) + X_{a,j} > y(a) + \epsilon M) = P(\min_j X_{a,j} > \epsilon M) \leq P(\min_j X_{a,j} > e \mathbb{E}[X_{a,j}])$. By the Markov's inequality on the right-hand side, we have that $P(\hat{y}(a) > y(a) + \epsilon M) \leq P(\min_j X_{a,j} > e \mathbb{E}[X_{a,j}]) \leq e^{-d} \leq \delta$. $\square$

B EDGE ANOMALIES-PROOFS

B.1 ANOEDGE-G

PROPOSITION 1. Time complexity of Algorithm 2 is $O(|\mathcal{E}| * n_r * n_b^2)$. Memory complexity of Algorithm 2 is $O(n_r * n_b^2)$.

PROOF. Procedure EDGE-SUBMATRIX-DENSITY removes rows (or columns) iteratively, and the total number of rows and columns that can be removed is $n_b + n_b - 2$. In each iteration, the approach performs the following three operations: (a) pick the row with minimum row-sum; (b) pick the column with minimum column-sum; (c) calculate density. We keep n_b -sized arrays for flagging removed rows (or columns), and for maintaining row-sums (or column-sums). Operations (a) and (b) take maximum n_b steps to pick and flag the row with minimum row-sum (or column-sum). Updating the column-sums (or rows-sums) based on the picked row (or column) again takes maximum n_b steps. Time complexity of (a) and (b) is therefore $O(n_b)$. Density is directly calculated based on subtracting the removed row-sum (or column-sum) and reducing the row-count (or column-count) from the earlier density value. Row-count and column-count are kept as separate variables. Therefore, the time complexity of the density calculation step is $O(1)$. Total time complexity of procedure EDGE-SUBMATRIX-DENSITY is $O((n_b + n_b - 2) * (n_b + n_b + 1)) = O(n_b^2)$.

Time complexity to initialize and decay the H-CMS data structure is $O(n_r * n_b^2)$. Temporal decay operation is applied whenever the timestamp changes, and not for every received edge. Update counts operation updates a matrix element value ($O(1)$ operation) for n_r matrices, and the time complexity of this step is $O(n_r)$. Anomaly score for each edge is based on the submatrix density computation

procedure which is $O(n_b^2)$; the time complexity of n_r matrices becomes $O(n_r * n_b^2)$. Therefore, the total time complexity of Algorithm 2 is $O(|\mathcal{E}| * (n_r + n_r * n_b^2)) = O(|\mathcal{E}| * n_r * n_b^2)$.

For procedure EDGE-SUBMATRIX-DENSITY, we keep an n_b -sized arrays to flag rows and columns that are part of the current submatrix, and to maintain row-sums and column-sums. Total memory complexity of EDGE-SUBMATRIX-DENSITY procedure is $O(4 * n_b) = O(n_b)$.

Memory complexity of H-CMS data structure is $O(n_r * n_b^2)$. Dense submatrix search and density computation procedure require $O(n_b)$ memory. For n_r matrices, this becomes $O(n_r * n_b)$. Therefore, the total memory complexity of Algorithm 2 is $O(n_r * n_b^2 + n_r * n_b) = O(n_r * n_b^2)$. $\square$

B.2 ANOEDGE-L

PROPOSITION 2. Time complexity of Algorithm 3 is $O(n_r * n_b^2 + |\mathcal{E}| * n_r * n_b)$. Memory complexity of Algorithm 3 is $O(n_r * n_b^2)$.

PROOF. As shown in Proposition 1, the time complexity of H-CMS is $O(n_r * n_b^2)$ and update operation is $O(n_r)$. Current submatrix (S_{cur}, T_{cur}) is updated based on *expand* and *condense* submatrix operations. (a) We keep an n_b -sized array to flag the current submatrix rows (or column), and also to maintain row-sums (or column-sums). Expand submatrix operation depends on the elements from row $h(u)$ and column $h(v)$, and the density is calculated by considering these elements, thus requiring maximum n_b steps. Upon addition of the row (or column), the dependent column-sums (or row-sums) are also updated taking maximum n_b steps. Time complexity of expand operation is therefore $O(n_b)$. (b) Condense submatrix operation removes rows and columns iteratively. A row (or column) elimination is performed by selecting the row (or column) with minimum row-sum (or column-sum) in $O(n_b)$ time. Removed row (or column) affects the dependent column-sums (or row-sums) and are updated in $O(n_b)$ time. Time complexity of a row (or column) removal is therefore $O(n_b)$. Condense submatrix removes rows (or columns) that were once added by the expand submatrix operation which in worse case is $O(|\mathcal{E}|)$.

Expand and condense submatrix operations are performed for n_r matrices. Likelihood score calculation depends on elements from row $h(u)$ and column $h(v)$, and takes $O(n_r * n_b)$ time for n_r matrices. Therefore, the total time complexity of Algorithm 3 is $O(n_r * n_b^2 + |\mathcal{E}| * n_r + |\mathcal{E}| * n_r * n_b + |\mathcal{E}| * n_r * n_b + |\mathcal{E}| * n_r * n_b) = O(n_r * n_b^2 + |\mathcal{E}| * n_r * n_b)$.

Memory complexity of the H-CMS data structure is $O(n_r * n_b^2)$. To keep current submatrix information, we utilize n_b -sized arrays similar to Proposition 1. For n_r matrices, submatrix information requires $O(n_r * n_b)$ memory. Hence, total memory complexity of Algorithm 3 is $O(n_r * n_b^2 + n_r * n_b) = O(n_r * n_b^2)$. $\square$

C GRAPH ANOMALIES-PROOFS

C.1 ANOGRAPH

PROPOSITION 3. Time complexity of Algorithm 4 is $O(|\mathcal{E}| * n_r * n_b^2 + |\mathcal{E}| * n_r)$. Memory complexity of Algorithm 4 is $O(n_r * n_b^2)$.

PROOF. Procedure ANOGRAPH-DENSITY iteratively removes row (or column) with minimum row-sum (or column-sum). Maximum

number of rows and columns that can be removed is $n_b + n_b - 2$. We keep n_b -sized arrays to store the current submatrix rows and columns, and row-sums and column-sums. At each iteration, selecting the row (or column) with minimum row-sum (or column-sum) takes $O(n_b)$ time, and updating the dependent row-sums (or column-sums) also $O(n_b)$ time. Density is calculated in $O(n_b)$ time based on the current submatrix row-sum and column-sum. Each iteration takes $O(n_b + n_b + n_b) = O(n_b)$ time. Hence, the total time complexity of ANOGRAPH-DENSITY procedure is $O((n_b + n_b - 2) * n_b) = O(n_b^2)$.

Initializing the H-CMS data structure takes $O(n_r * n_b^2)$ time. When a graph arrives, ANOGRAPH: (a) resets counts that take $O(n_r * n_b^2)$ time; (b) updates counts taking $O(1)$ time for every edge update; (c) computes submatrix density that follows from procedure ANOGRAPH-DENSITY and takes $O(n_b^2)$ time. Each of these operations is applied for n_r matrices. Therefore, the total time complexity of Algorithm 4 is $O(n_r * n_b^2 + |\mathcal{E}| * n_r * n_b^2 + |\mathcal{E}| * n_r + |\mathcal{E}| * n_r * n_b^2) = O(|\mathcal{E}| * n_r * n_b^2 + |\mathcal{E}| * n_r)$, where $|\mathcal{E}|$ is the total number of edges over graphs $\mathcal{G}$.

For procedure ANOGRAPH-DENSITY, we keep n_b -sized array to flag rows and columns that are part of the current submatrix, and to maintain row-sums and column-sums. Hence, memory complexity of ANOGRAPH-DENSITY procedure is $O(4 * n_b) = O(n_b)$.

H-CMS data structure requires $O(n_r * n_b^2)$ memory. Density computation relies on ANOGRAPH-DENSITY procedure, and takes $O(n_b)$ memory. Therefore, the total memory complexity of Algorithm 4 is $O(n_r * n_b^2)$. $\square$

C.2 Proofs: ANOGRAPH-K

PROPOSITION 4. *Time complexity of Algorithm 5 is $O(|\mathcal{E}| * K * n_r * n_b^2 + |\mathcal{E}| * n_r)$. Memory complexity of Algorithm 5 is $O(n_r * n_b^2)$.*

PROOF. Relevant operations in Procedure ANOGRAPH-K-DENSITY directly follow from EDGE-SUBMATRIX-DENSITY procedure, which has $O(n_b^2)$ time complexity. EDGE-SUBMATRIX-DENSITY procedure is called K times, therefore, the total time complexity of ANOGRAPH-K-DENSITY procedure is $O(K * n_b^2)$.

For Algorithm 5, we initialize an H-CMS data structure that takes $O(n_r * n_b^2)$ time. When a graph arrives, ANOGRAPH-K: (a) resets counts that take $O(n_r * n_b^2)$ time; (b) updates counts taking $O(1)$ time for every edge update; (c) computes submatrix density that follows from procedure ANOGRAPH-K-DENSITY and takes $O(K * n_b^2)$ time. Each of these operations is applied for n_r matrices. Therefore, the total time complexity of Algorithm 5 is $O(n_r * n_b^2 + |\mathcal{E}| * K * n_r * n_b^2 + |\mathcal{E}| * n_r + |\mathcal{E}| * n_r * n_b^2) = O(|\mathcal{E}| * K * n_r * n_b^2 + |\mathcal{E}| * n_r)$, where $|\mathcal{E}|$ is the total number of edges over graphs $\mathcal{G}$.

The density of K submatrices is computed independently, and the memory complexity of Algorithm procedure ANOGRAPH-K-DENSITY is the same as the memory complexity of EDGE-SUBMATRIX-DENSITY procedure i.e. $O(n_b)$.

Maintaining the H-CMS data structure requires $O(n_r * n_b^2)$ memory. Density computation relies on ANOGRAPH-K-DENSITY procedure, and it requires $O(n_b)$ memory. Therefore, the total memory complexity of Algorithm 5 is $O(n_r * n_b^2)$. $\square$

D EXPERIMENTAL SETUP

All experiments are carried out on a 2.4GHz Intel Core i9 processor, 32GB RAM, running OS X 10.15.3. For our approach, we keep $n_r = 2$ and $n_b = 32$ to have a fair comparison to MIDAS which uses $n_b^2 = 1024$ buckets. Temporal decay factor $\alpha = 0.9$ for Algorithms 2 and 3. We keep $K = 5$ for Algorithm 5. AUC for graph anomalies is shown with edge thresholds as 50 for DARPA and 100 for other datasets. Time window is taken as 30 minutes for DARPA and 60 minutes for other datasets.

E BASELINES

We use open-source implementations of DENSESTREAM [1], SEDANSPOT [11], MIDAS-R [3] (C++), PENminer [12], F-FADE [13], DENSEALERT [1], and ANOMRANK [14] provided by the authors, following parameter settings as suggested in the original paper. For SPOTLIGHT [2], we used open-sourced implementations of Random Cut Forest [66] and Carter Wegman hashing [67].

E.1 Edge Anomalies

- (1) SEDANSPOT: sample_size = 10000, num_walk = 50, restart_prob 0.15
- (2) MIDAS: The size of CMSs is 2 rows by 1024 columns for all the tests. For MIDAS-R, the decay factor $\alpha = 0.6$.
- (3) PENminer: ws = 1, ms = 1, view = id, alpha = 1, beta = 1, gamma = 1
- (4) DENSESTREAM: We keep default parameters, i.e., order = 3.
- (5) F-FADE: embedding_size = 200, W_upd = 720, T_th = 120, alpha = 0.999, M = 100
For t_setup, we always use the timestamp value at the 10th percentile of the dataset.

E.2 Graph Anomalies

- (1) SPOTLIGHT: K = 50, p = 0.2, q = 0.2
- (2) DENSEALERT: We keep default parameters, i.e., order = 3 and window=60.
- (3) ANOMRANK: We keep default parameters, i.e., damping factor c = 0.5, and L1 changes of node score vectors threshold epsilon = 10^{-3} . We keep $1/4^{th}$ number of graphs for initializing mean/variance as mentioned in the respective paper.